Unit – 2: Cursor Management

Prepared By: Prof. Meghna K. Bhatt

fppt.com

Course content

- Introduction to Cursor
- Types of Cursors:
  - Implicit Cursor,
    - Cursor Attributes,
  - Explicit Cursors:
    - Cursor declaration,
    - opening,
    - fetching data from cursor,
    - closing,
- Cursor FOR loop
- Parameterized Cursor

fppt.com

Overview of Cursor

- A cursor is a temporary work area created in the system memory when a SQL statement is executed.
- A cursor contains information on a select statement and the rows of data accessed by it.
- This temporary work area is used to store the data retrieved from the database, and perform intermediate operations before output is displayed to the client.
- A cursor can hold more than one row, but can process only one row at a time.

fppt.com

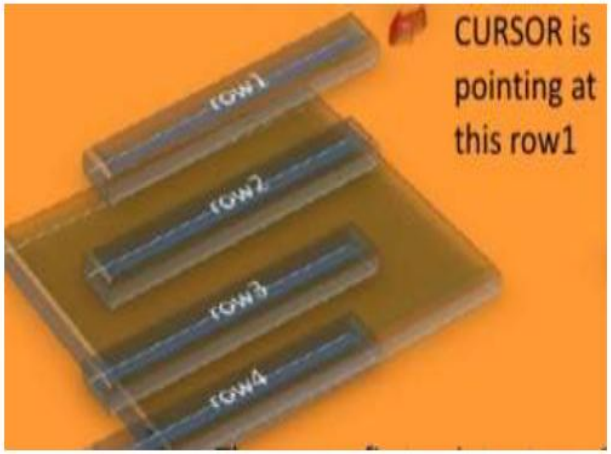
Introduction to Cursor

- Cursor is a memory area which store records written by SQL query.
- It is mainly used to perform operations on more then one row in PL block.
- The set of rows returned by a cursor is called an Active Data Set (Result Set).
- The row that is being processes is called the Current Row.
- The Current Row inside the Active Data Set (Result Set) is identified by the cursor, which allows the individual processing of each of them.
- The data that is stored in the cursor is called the **Active Data Set.**

fppt.com

Need for Cursors?

- Cursor is a mechanism that provides a way to select multiple rows of data from the database and then process each row individually inside a PL/SQL program.

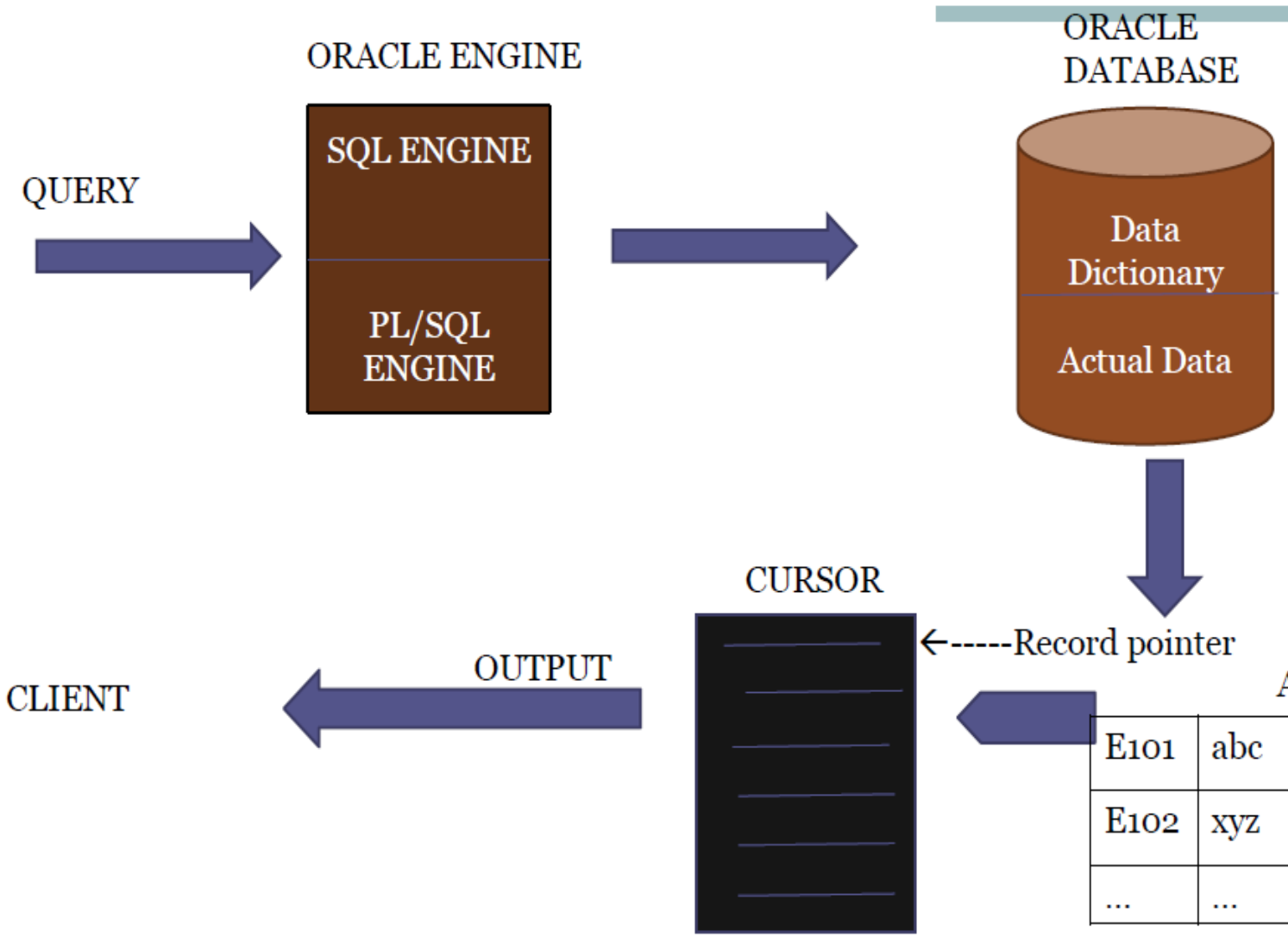




- The cursor first points at row1 and once it is processed it then advances to row2 and so on.

fppt.com

How Cursor Works?



Cursor data type

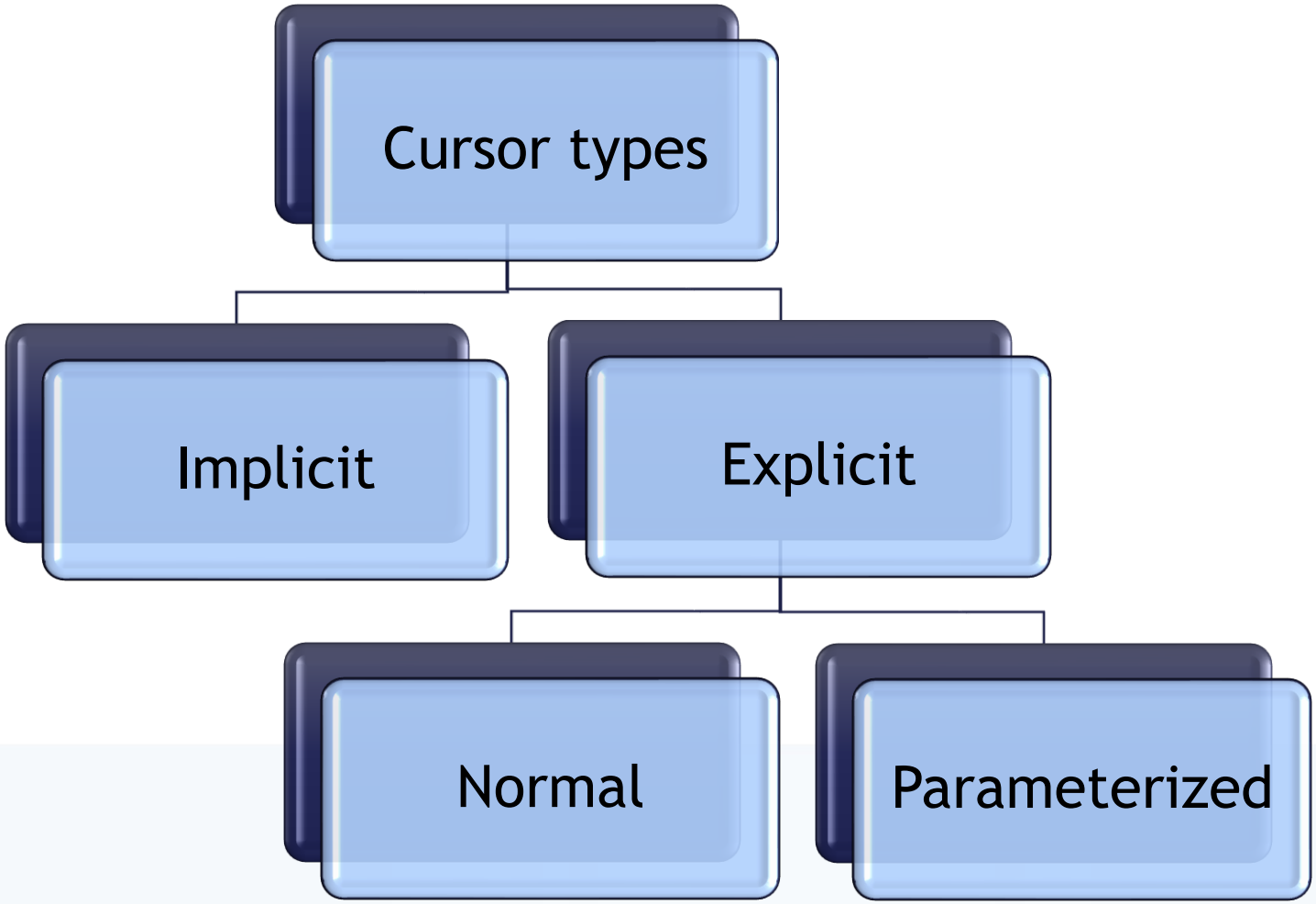
- There are two types of cursor data type:
  - %TYPE
    - Store column/field inside a variable then %TYPE will be used.
    - syntax

      CUR_VARI_NAME <Tablename>.<Fieldname>%TYPE;
  - % ROWTYPE
    - Store Records inside a variable then %TYPE will be used.
    - syntax

      CUR_VARI_NAME <Tablename>%ROWTYPE;

fppt.com

Types of Cursor



fppt.com

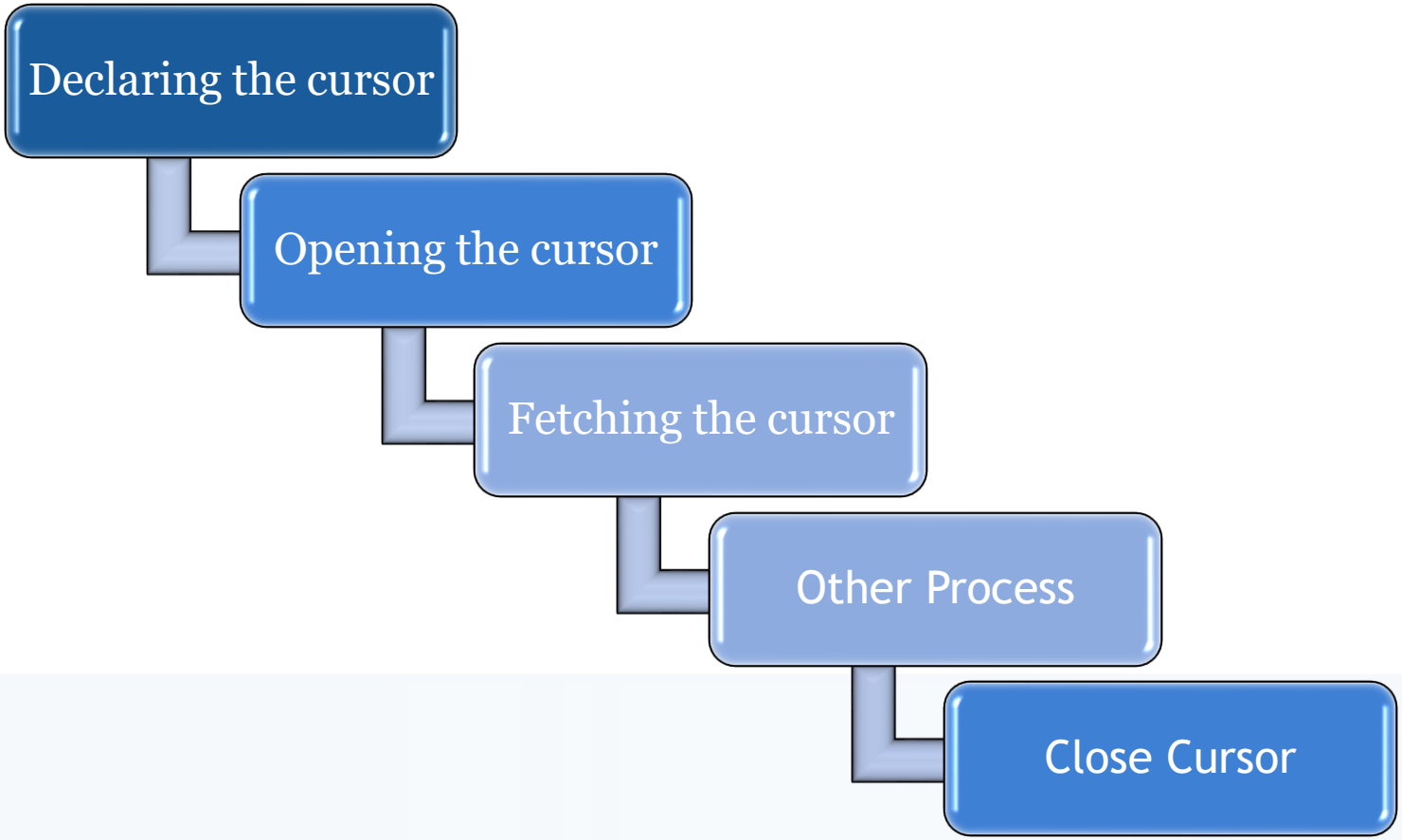
Explicit cursor

- Its called user define cursor.
- Explicit cursors are programmer defined cursors for gaining more control over the context area.
- An explicit cursor should be defined in the declaration section of the PL/SQL Block.
- It is created on a SELECT Statement which returns more than one row.

fppt.com

Explicit Cursors

- **Explicit cursor involves five steps:**



fppt.com

Explicit Cursors

- **Explicit cursor involves four steps:**



- Create a named SQL area
- Identify the active set
- Load the current row into variables
- Test for existing rows
- Return to FETCH if rows found
- Release the active set

fppt.com

Cursor Declaration

- Cursor is declare in declaration part of PL/SQL block
- Syntax:

```
Cursor <cursor_name> IS Select */<fields> FROM <Table_name> ;
```

- Example :

```
Cursor stud_Cur IS Select name, bdate  FROM stud ;
```

fppt.com

Open Cursor

- A user define / explicit cursor must be open before start process.
- Cursor must be open in execution section of PL/SQL block
- Syntax:

```
Open <cursor_name>  ;
```

- Example :

```
Cursor stud_Cur;
```

fppt.com

Fetch date from Cursor

- Fetch takes the data from cursor & store into memory variables and process them one by one using loop.
- Syntax:

```
Fetch <cursor_name> INTO cur_v1, cur_v2….. ;
```

- Example:

```
Cursor stud_Cur into xrlno,xbdate;
```

fppt.com

Close Cursor

- After the FETCH loop exits, the cursor must be closed by Close statement.
- At the end it releases the used memory
- Syntax:

```
Close   <cursor_name;
```

- Example

```
Close stud_Cur;
```

fppt.com

Explicit Cursors Attributes



fppt.com

Explicit Cursors Attributes

Attribute Name	Description
%ISOPEN	Returns TRUE if cursor is open, FALSE if cursor is closed. CursorName%ISOPEN is used to access it.
%FOUND	Returns TRUE if successful fetch has been executed, FALSE if no row was returned. CursorName%FOUND is used to access it.
%NOTFOUND	Return TRUE if no row was returned, FALSE if successful fetch has been executed. CursorName%NOTFOUND is used to access it.
%ROWCOUNT	Returns the number of rows affected by the query. CursorName%ROWCOUNT is used to access it.

Explicit Cursor

- **Explicit Cursor (Example):**

```
SET SERVEROUTPUT ON
DECLARE
    c_id customers.id%type;
    c_name customers.name%type;
    c_addr customers.address%type;
    CURSOR c_customers IS SELECT id, name, address FROM customers;
BEGIN
    OPEN c_customers;
    LOOP
        FETCH c_customers into c_id, c_name, c_addr;
        EXIT WHEN c_customers%notfound;
        dbms_output.put_line(c_id || ' ' || c_name || ' ' || c_addr);
    END LOOP;
    CLOSE c_customers;
END;
/
```

fppt.com

IMPLICIT CURSOR

- These are created by default when DML statements like, INSERT, UPDATE, and DELETE statements are executed.
- The user is not aware of this happening & will not be able to control or process the information.
- When an implicit cursor is working, DBMS performs the open, fetches and close operations automatically.

fppt.com

Implicit Cursors Attributes



fppt.com

Implicit Cursors

Attribute Name	Description
%ISOPEN	Returns TRUE if cursor is open, FALSE if cursor is closed. SQL%ISOPEN always returns FALSE.
%FOUND	Returns TRUE if successful fetch has been executed, FALSE if no row was returned. SQL%FOUND is used to access it.
%NOTFOUND	Return TRUE if no row was returned, FALSE if successful fetch has been executed. SQL%NOTFOUND is used to access it.
%ROWCOUNT	Returns the number of rows affected by the query. SQL%ROWCOUNT is used to access it.

Implicit Cursors

- **Implicit Cursor (Example):**
- Write a pl/SQL program to update the salary of customer by Rs 500.
- The data of customers table before update operation:

```
Select * from customers;

+----+---------+-----+-----------+---------+
| ID | NAME | AGE | ADDRESS | SALARY |
+----+---------+-----+-----------+---------+
| 1 | Ramesh | 32 | Ahmedabad | 2000.00 |
| 2 | Khilan  | 25 | Delhi     | 1500.00 |
| 3 | kaushik | 23 | Kota      | 2000.00 |
| 4 | Chaitali | 25 | Mumbai   | 6500.00 |
| 5 | Hardik  | 27 | Bhopal    | 8500.00 |
| 6 | Komal   | 22| MP         | 4500.00 |
+----+---------+-----+-----------+---------+
```

fppt.com

Implicit Cursors

- **Implicit Cursor (Example):**

```
SET SERVEROUTPUT ON
DECLARE
    total_rows number(2):=0;
BEGIN
    UPDATE customers SET salary = salary + 500;
    IF sql%notfound THEN
        dbms_output.put_line('no customers selected');
    ELSIF sql%found THEN
        total_rows:=sql%rowcount;
        dbms_output.put_line(total_rows || 'customers salary
    updated');
    END IF;
END;
/
```

fppt.com

Implicit Cursors

- **Implicit Cursor (Example Output):**
- 6 customers salary updated.
- PL/SQI procedure successfully completed
- Data of

Select * from customers;

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2500.00
2	Khilan	25	Delhi	2000.00
3	kaushik	23	Kota	2500.00
4	Chaitali	25	Mumbai	7000.00
5	Hardik	27	Bhopal	9000.00
6	Komal	22	MP	5000.00

fppt.com

DIFFERENCE BETWEEN IMPLICIT & EXPLICIT

fppt.com

Difference between Implicit & explicit curs

IMPLICIT CURSOR	EXPLICIT CURSOR
If Oracle engine for its internal processing has opened a cursor they are known as Implicit Cursors.	A user can also open a cursor for processing data as required. Such user-defined cursors are known as Explicit Cursors.
Oracle by default creates the cursor with name SQL.	User can create the explicit cursor with any name and also can changes the name of the cursor.
We have to put SQL prefix before accessing any attributes of the Implicit Cursor.	If we want to use any Explicit Cursors attributes then we have to put user defined cursor's name as a prefix.
User cannot require OPENING and CLOSING of the implicit cursor.	User must have to OPEN the cursor for fetching the data and CLOSE the cursor after completion of operation.
We cannot use the FETCH command with Implicit Cursor.	We have to use the FETCH command with Explicit Cursor.
There are no other types of implicit cursors.	There are two types of explicit cursors are there: (i) Normal Cursor. (ii) Parameterized Cursor.

Explicit cursor

- Explicit cursors are programmer defined cursors for gaining more control over the context area.
- An explicit cursor should be defined in the declaration section of the PL/SQL Block.
- It is created on a SELECT Statement which returns more than one row.

fppt.com

Explicit Cursor

- **Explicit Cursor (Example Output):**

```
1    Ramesh    Ahmedabad
2    Khilan    Delhi
3    kaushik   Kota
4    Chaitali  Mumbai
5    Hardik    Bhopal
6    Komal     MP

PL/SQL procedure successfully completed.
```

fppt.com

Cursor FOR Loop

- A Cursor FOR LOOP is a loop meant for the cursor which automatically checks for the row count and exits the loop
- when all the data stored in the cursor is iterated.
- A cursor FOR loop automatically does the following:
  - Implicitly declares its loop index as a %rowtype record
  - Opens the cursor
  - Retrieves the record from the cursor for each iteration
  - Closes the cursor after processing all the records.
  - A cursor can also be closed by using EXIT or GOTO statements.

fppt.com

Cursor FOR Loop

- **Cursor FOR Loop Syntax:**

FOR <VARIABLE> IN <CURSOR_NAME> LOOP

<STATEMENTS>;

END LOOP;

```
FOR record_name IN cursor_name LOOP
   statement1;
   statement2;
   . . .
END LOOP;
```

fppt.com

Cursor FOR Loop

- **Explicit Cursor (Example of Cursor FOR Loop):**

```
SET SERVEROUTPUT ON

DECLARE
    CURSOR c1 IS SELECT last_name, job_id FROM employee job_id LIKE
    '%CLERK%‘ AND manager_id > 120 ORDER BY last_name;
BEGIN
    FOR item IN c1 LOOP
            DBMS_OUTPUT.PUT_LINE ('Name = ' || item.last_name || ', Job =
    '       ||item.job_id);
    END LOOP;
END;
/
```

fppt.com

Cursor FOR Loop

- **Output: (Example of Cursor FOR Loop):**

```
Name = Atkinson, Job = ST_CLERK
Name = Bell, Job = SH_CLERK
Name = Bissot, Job = ST_CLERK
 ...
Name = Walsh, Job = SH_CLERK
```

fppt.com

Parameterized Cursor

- Parameterized cursor pass the parameters into a cursor and use them in to query.
- Define only data type of parameter and not need to define it's length.
- We can only pass values to the cursor and cannot pass values out of the cursor through parameters.
- The scope of the cursor parameters is local to the cursor.

fppt.com

Parameterized Cursor

- **Parameterized Cursor (Syntax):**

```
CURSOR cursor_name
  [(parameter_name datatype, ...)]
IS
  select_statement;
```

CURSOR <CURSOR_NAME>
[(PARAMETER1 DATATYPE, PARMETER2 DATATYPE,....)]
IS <SELECT STATEMENT>;

- **Syntax to open Parameterized Cursor:**

```
OPEN  cursor_name(parameter_value,.....) ;
```

fppt.com

Parameterized Cursor

- **Parameterized Cursor (Example):**

```
CURSOR c2 (subject_id_in IN varchar) IS
SELECT course_number FROM courses_tbl
WHERE subject_id = subject_id_in;

OPEN C2(1);
```

Output:

- The result set of this cursor is all course_numbers whose
- subject_id matches the subject_id passed to the cursor via the parameter.

fppt.com

EXAMPLE

```
DECLARE
CURSOR stu_cursor(xgrade CHAR) IS
    SELECT roll_no, name FROM student WHERE grade = xgrade;

BEGIN
  -- FOR loop automatically handles open/fetch/close
  FOR s IN stu_cursor('A') LOOP
    DBMS_OUTPUT.PUT_LINE('Roll No: ' || s.roll_no || ', Name: '
|| s.name);
  END LOOP;
END;
/
```

fppt.com

Advantages

- Using Cursor we can perform row by row processing .
- We can perform row wise validation or operations on each row.
- Allow application to access and move around in a set of data rows, rather then merely retrieve a complete result set.

fppt.com

Disadvantages

- Uses more resources.
- Speed and performance issues.
- Increased network roundtrip.

fppt.com